

GPU-Accelerated Community Detection: Improving the Louvain Algorithm through Hybrid and Adaptive Methods

Aniket Jain (M25CS002), Ayush Pal (M25CS004), Ishan Bhati (M25CS006), Shrinidhi Narwriya (M25CS014)

Department of Computer Science and Engineering, Indian Institute of Technology Bhilai

CS516/CSL516: Parallelization of Programs

ABSTRACT

Detecting communities within large graphs sits at the heart of many real-world analytics tasks, from mapping influence in social networks to pinpointing functional clusters in biological systems. Among the many algorithms designed for this purpose, the Louvain method has become a go-to choice because of how well it scales without sacrificing partition quality. A recent GPU adaptation of the algorithm, GALA (PPoPP 2025), pushes performance further by combining a theoretically-grounded pruning scheme with hardware-aware kernel design. Even so, we found that three concrete bottlenecks cap its throughput: poor memory-access locality caused by arbitrary vertex numbering, a cold-start problem stemming from singleton community initialisation, and a kernel-selection policy that is oblivious to the actual degree profile of the input graph. We tackle all three with purpose-built enhancements: **(N1) graph reordering** via BFS traversal order and degree sorting to coalesce GPU memory accesses; **(N2) a GPU-resident LPA warm-start** that feeds Louvain a meaningful starting partition instead of singletons; and **(N3) an adaptive kernel threshold** derived from the 75th-percentile degree at run-time. Running on the *com-Amazon* co-purchase graph (334,863 vertices, 925,872 edges) on an NVIDIA T4 GPU, degree-sorted reordering tops out at **1.76×** speedup under the MG pruning mode, while BFS reordering hits **1.67×** with zero modularity loss.

I. INTRODUCTION

Graphs show up everywhere in modern computing—social platforms, biological databases, logistics networks, and the web itself all lend themselves naturally to a graph representation. One of the first questions analysts ask about any such graph is whether its vertices organise into tightly-knit groups, loosely connected to one another but dense within. Answering that question is the community-detection problem [14], and it has real consequences: spotting echo chambers in online discourse [15], uncovering latent structure in spatial networks [16], and guiding recommendation systems all depend on solving it well.

The Louvain algorithm [1] approaches the problem by greedily maximising a quality measure called modularity. It alternates between two phases: first, each vertex looks at its neighbours and migrates to whichever community offers the greatest modularity gain ΔQ ; second, each community is collapsed into a single super-vertex, shrinking the graph for the next round. This greedy, hierarchical strategy converges quickly in practice and produces high-quality partitions even on graphs with millions of edges, which explains why it remains widely used.

Where existing GPU implementations fall short. Carry-

ing the Louvain algorithm onto a GPU is non-trivial. Early efforts [5, 6] mapped each vertex’s community-selection step onto a group of threads, but stored all intermediate weights in global memory—a design that saturates bandwidth and kills throughput. GALA [2] makes two significant advances over this baseline. First, it introduces a modularity-gain-based pruning rule (MG) that can prove a vertex will not move in the current round without actually executing the expensive DecideAndMove scan, and does so without ever producing a wrong answer. Second, it splits the workload between a warp-level shuffle kernel for low-degree vertices and a block-level hierarchical hash kernel for high-degree ones, so each vertex gets the memory tier that suits it best. Despite these advances, our profiling identified three remaining gaps:

- **Memory locality:** vertices retain their original IDs in the CSR layout, so the neighbours of a single vertex can be scattered across GPU memory, preventing coalesced warp access.
- **Cold initialization:** because every vertex starts in its own singleton community, the first round of Louvain must process essentially the whole graph with almost no pruning benefit.
- **Static kernel dispatch:** GALA decides which kernel to use based on fixed degree thresholds (4, 8, 16, 32, 128, 1024) baked in at compile time, regardless of whether those values suit the actual input graph.

What we built. Rather than redesigning GALA from scratch, we layered three lightweight enhancements on top of it, each targeting one of the gaps above:

1. **Graph Reordering (N1):** a preprocessing step that re-assigns vertex IDs—either in BFS traversal order or by ascending degree—so that related vertices sit close together in memory.
2. **GPU LPA Warm-Start (N2):** three rounds of Label Propagation run entirely on the GPU before Louvain begins, giving the algorithm a head start instead of a blank slate.
3. **Adaptive Kernel Threshold (N3):** the 75th-percentile degree of the input graph is computed at load time and used to set the warp/block dispatch boundary, replacing the static compile-time constant.

II. APPROACH / ALGORITHM

A. Background: GALA

We work on undirected weighted graphs $G = (V, E, w)$. Modularity measures how much the edge density inside communities

exceeds what you would expect at random:

$$Q = \sum_{C \in \mathcal{C}} \left(\frac{D_C(C)}{2|E|} - \left(\frac{D_V(C)}{2|E|} \right)^2 \right) \quad (1)$$

where $D_C(C)$ counts total edge weight inside community C and $D_V(C)$ is the sum of degrees of its members. Each iteration of Louvain asks: by how much would modularity change if vertex v moved into community C ? That gain is:

$$\Delta Q_{v \rightarrow C} = \frac{1}{|E|} \left(d_C(v) - \frac{D_V(C) \cdot d(v)}{2|E|} \right) \quad (2)$$

GALA’s MG strategy derives a provable upper bound on the best ΔQ any move could offer. If that bound is non-positive the vertex cannot benefit from moving, so its `DecideAndMove` call is skipped entirely—with a theoretical guarantee of no missed improvements [2]. One implementation subtlety worth noting: GALA’s `degrees[]` array stores a running cumulative sum rather than per-vertex counts, so the actual degree of vertex v is `degrees[v] - degrees[v-1]`. Any preprocessing that touches vertex IDs must rebuild this prefix sum correctly or the kernel launch parameters will be wrong.

B. N1 — Graph Reordering for Memory Locality

A GPU warp reads memory most efficiently when all 32 of its threads target a contiguous 128-byte block. Real-world graphs rarely cooperate: a vertex’s neighbour list can span widely separated cache lines simply because vertex IDs were assigned in an arbitrary order. We address this by renumbering vertices before the graph is handed to GALA, then reconstructing the CSR adjacency arrays under the new numbering—making sure the prefix-sum `degrees[]` is rebuilt correctly alongside the neighbour list.

BFS-based reordering (N1a). We start a breadth-first traversal from vertex 0 and assign fresh IDs in the order vertices are discovered. Disconnected components are handled by picking the next unvisited vertex as a new source whenever the queue empties. The intuition is that vertices close together in the graph tend to share neighbour lists; placing them close in memory increases the chance that a single cache-line fetch serves multiple threads in the same warp. In our experiments this cut round-0 `DecideAndMove` time from 32.07 ms down to 15.02 ms—a 53% reduction.

Degree-sorted reordering (N1b). Here vertices are sorted by degree in ascending order and renumbered accordingly, so the lowest-degree vertices occupy the earliest positions in the array. This matters because GALA already splits work by degree: small-degree vertices go to the shuffle kernel and large-degree vertices to the hash kernel. By making the degree ordering match the physical layout, threads executing the same kernel path access a contiguous region of memory, reducing warp divergence and bank conflicts in the shared-memory hash table. Round-0 total time fell from 82.60 ms to 37.34 ms, and weight-update time dropped from 15.54 ms to just 8.90 ms.

C. N2 — GPU LPA Warm-Start

Starting Louvain from singleton communities means that in round 0 almost every vertex is active, and pruning can do little to reduce the load. We sidestep this cold-start problem by running a short burst of Label Propagation on the GPU before the Louvain loop begins.

Each thread takes ownership of one vertex, scans its adjacency list, tallies incoming edge weight for each community label it encounters using a thread-local table capped at 128 entries, and then switches to the label with the greatest total weight. When two labels tie, the smaller numerical ID wins, keeping the result deterministic. We use 256 threads per block and fix the number of LPA rounds at three, a choice that balances warm-start quality against the roughly 13.8 ms overhead each execution adds. After LPA finishes, many vertices are already sitting in a sensible community, so the first Louvain iteration encounters a much smaller active set.

D. N3 — Adaptive Kernel Threshold

GALA’s compile-time degree buckets were calibrated on a set of benchmark graphs used in the original paper. There is no reason to expect those same cut-points to be optimal for every input. Our solution is to inspect the input graph at load time, compute the 75th-percentile degree p_{75} , and use a clamped version of that value as the actual warp/block boundary:

$$t_{\text{adaptive}} = \text{clamp}(p_{75}, 4, 128) \quad (3)$$

For com-Amazon, p_{75} works out to 6, which means three-quarters of all vertices have degree at most 6. GALA’s static threshold of 32 would send all of them to the hash-based kernel; our threshold correctly routes them to the lighter shuffle kernel, which operates entirely out of warp registers with no shared memory overhead. When used alongside degree-sorted reordering, vertices within the same degree bucket also form a contiguous segment in memory, letting each kernel sweep a clean, coalesced region of the CSR arrays.

III. EVALUATION METHODOLOGY

A. Hardware Platform

We ran all experiments on a Google Colaboratory instance backed by an **NVIDIA Tesla T4 GPU** (Turing, sm_75, 16 GB GDDR6, 320 GB/s peak bandwidth, 2560 CUDA cores, 70 W TDP). The host machine exposes an **Intel Xeon E5-2699 v4** processor (dual-socket, 22 cores per socket, 2.2 GHz base, 55 MB LLC) with 12.7 GB of RAM available to the runtime. All GPU code was compiled with NVCC 12.8 targeting `-arch=sm_75`; host code used GCC 11.4 with `-O3 -std=c++17`.

B. Dataset

Our evaluation uses the **com-Amazon** undirected co-purchase graph obtained from the Stanford SNAP repository [13], which is a widely adopted benchmark for community-detection research. Key properties are listed in Table 1.

C. Baselines and Pruning Modes

We treat the **unmodified GALA binary** [2] run on the original graph as our baseline. Every novelty is evaluated under all four

Table 1: Dataset Statistics

Property	Value
Vertices	334,863
Edges	925,872
Graph type	Undirected, unweighted
Source	SNAP com-Amazon

Table 2: Average Time, Speedup & Modularity (com-Amazon, T4 GPU)

Method	Avg					
	Avg Mod.	Time (ms)	Min (ms)	Max (ms)	Avg Spd.	Best Spd.
Baseline GALA	0.9224	833.8	602.0	1007.9	1.00×	1.00×
N1a: BFS Reorder	0.9225	598.6	590.6	605.1	1.39×	1.67×
N1b: Deg-Sort	0.9219	589.5	569.1	616.0	1.42×	1.76×
N2: LPA Warm-Start	0.9224	693.1	636.4	787.9	1.19×	1.38×
N3: Adaptive	0.9219	650.0	599.0	715.5	1.27×	1.46×
Combined	0.9225	709.9	667.4	778.8	1.17×	1.35×

pruning modes that GALA supports, so that no single mode can hide a regression:

- **p=0 (MG):** the modularity-gain strategy—provably lossless, zero false negatives.
- **p=1 (RM):** relaxed movement—faster pruning at the cost of roughly 0.001 modularity.
- **p=2 (Vite):** probabilistic pruning with $\alpha = 0.25$.
- **p=3 (MG+RM):** both MG and RM applied together.

Average and best-case speedup figures are reported across the four modes.

D. Performance Metrics

Three numbers capture performance in our study: **total wall-clock time (ms)**, measured from graph load through algorithm termination and including all preprocessing; **speedup**, computed as the ratio of baseline time to optimized time under the same pruning mode; and **modularity** $Q \in [-1, 1]$, which scores partition quality—higher values indicate better community structure.

IV. EXPERIMENTAL RESULTS

Tables 2–4 summarise our findings. Table 2 gives the headline numbers averaged over all four pruning modes. Table 3 breaks speedup down per-mode, and Table 4 shows where the time goes inside round 0 under the MG pruning strategy.

A. N1: Graph Reordering

Reordering the graph turned out to be our single strongest lever. **BFS reordering (N1a)** brought average execution time down from 833.8 ms to 598.6 ms, a **1.39×** average gain that peaks at **1.67×** on the MG pruning mode. Notably, modularity actually ticked up by a hair to 0.9225, reassuring us that we did not trade quality for speed. The improvement traces back almost entirely to the DecideAndMove kernel: as shown in Table 4, its round-0 runtime shrank from 32.07 ms to 15.02 ms. Once vertices with shared neighbours are stored adjacently in the CSR

Table 3: Speedup by Pruning Mode

Method	p=0 MG	p=1 RM	p=2 Vite	p=3 MG+RM
Baseline GALA	1.00×	1.00×	1.00×	1.00×
N1a: BFS Reorder	1.67×	1.62×	1.28×	1.00×
N1b: Deg-Sort	1.76×	1.68×	1.28×	0.98×
N2: LPA Warm-Start	1.28×	1.38×	1.17×	0.95×
N3: Adaptive	1.41×	1.46×	1.22×	1.01×
Combined	1.29×	1.35×	1.12×	0.90×

Table 4: Round-0 Kernel Timing Breakdown (p=0 MG run)

Method	DecideAndMove (ms)	Weight Update (ms)	Round-0 Total (ms)
Baseline GALA	32.07	15.54	82.60
N1a: BFS Reorder	15.02	13.75	51.29
N1b: Deg-Sort	15.09	8.90	37.34
N2: LPA Warm-Start	28.20	13.70	64.41
N3: Adaptive	15.56	10.24	45.84
Combined	16.97	15.39	58.99

arrays, a single warp accessing one vertex’s neighbourhood also pre-fetches relevant data for the next vertex in the active set, so fewer global memory transactions are needed.

Degree-sorted reordering (N1b) pushed the average time further down to 589.5 ms and delivered a peak of **1.76×** at MG. The weight-update kernel benefited even more noticeably here: its round-0 time fell from 15.54 ms to 8.90 ms, because vertices sharing the same degree bucket also tend to interact with the same communities, reducing atomic collisions in the shared-memory hash table. Modularity dropped slightly to 0.9219—not a flaw in the optimisation, but a natural consequence of processing vertices in a different order; the greedy Louvain step simply converges to a nearby but distinct local optimum. The one place N1b underperforms is p=3 MG+RM (0.98×): the combined pruning strategy’s movement-prediction logic seems to be sensitive to the vertex ordering, causing a few extra vertices to be reactivated that would otherwise have stayed pruned.

B. N2: GPU LPA Warm-Start

Running three rounds of LPA before Louvain gave us a **1.19×** average speedup, reaching **1.38×** at p=1 RM. Modularity was unchanged at 0.9224, matching the baseline exactly under all modes, which confirms that the warm start does not steer Louvain toward a worse solution.

The gain is clearest under RM pruning. RM decides whether to skip a vertex based on whether its community ID changed in the last round; a richer initial partition means fewer vertices flip IDs in round 0, so RM’s active set is smaller right from the start. MG, on the other hand, builds its pruning criterion from a mathematical bound that is independent of the starting partition, so it benefits less. The 13.8 ms LPA overhead is real but more than recovered under MG and RM modes. Under p=3 MG+RM the story reverses: combined pruning is aggressive enough that the LPA labels occasionally conflict with the movement signals it relies on, leading to a net 0.95× slowdown rather than a speedup.

C. N3: Adaptive Kernel Threshold

Setting the kernel dispatch threshold from the data rather than from a fixed constant yielded a $1.27\times$ average speedup and a best result of $1.46\times$ at $p=1$ RM. For com-Amazon the 75th-percentile degree evaluates to 6. That means GALA’s hardcoded threshold of 32 had been pushing 75% of all vertices into the hash-based kernel, which requires shared-memory allocation and synchronisation, even though those vertices are small enough to fit comfortably in warp registers. Lowering the cut-off to 6 routes the right vertices to the shuffle kernel and noticeably reduces occupancy pressure on the shared-memory pool.

D. Combined Configuration

Stacking all three novelties together produced a $1.17\times$ average speedup ($1.35\times$ best at $p=1$ RM), with modularity holding at 0.9225—the highest value we observed across the entire study. The headline number being lower than N1b alone is not a surprise: the combined run uses BFS reordering (N1a) rather than degree sorting (N1b) because degree sorting and LPA interact in ways that occasionally hurt MG+RM, and the 13.8 ms LPA overhead has to be absorbed in every run. At $p=3$ MG+RM the three novelties together register $0.90\times$, the only configuration-mode pair in our study where an optimised run is slower than baseline—a clear sign that aggressive pruning and structural reordering need more careful co-design.

E. Modularity Analysis

Every configuration we tested stayed within 0.0015 of the baseline modularity of 0.9224. Under $p=0$ MG, which guarantees that no improving vertex move is ever skipped, N2 reproduces the baseline score exactly (0.9224). The 0.9219 reading seen for N1b and N3 comes from those methods changing the order in which vertices are considered during DecideAndMove; because Louvain is greedy, two different traversal orders can converge to two different local optima, and a gap of roughly 5×10^{-4} in modularity is entirely typical for this family of algorithms on this class of graph.

V. RELATED WORK

CPU-based Louvain. Blondel et al. [1] introduced the original two-phase algorithm and showed it could handle networks of millions of nodes in minutes. Grappolo [3] recast the local-move phase as a bulk-synchronous parallel computation, a framing that almost all subsequent GPU implementations have inherited. Vite [4] took the next step by distributing the graph across multiple nodes and adding a probabilistic pruning rule to avoid processing nearly-stable vertices.

GPU implementations. Moving Louvain to the GPU requires solving a harder memory-management problem because the DecideAndMove function needs to aggregate edge weights by community label on the fly. Earlier work [7] stored everything in global memory and paid a heavy latency penalty. Naim et al. [5] introduced a shared-memory hash table for smaller workloads, and Bhowmik and Vadhiyar [6] split the algorithm between CPU and GPU to manage high-degree vertices separately. GALA [2] unified these ideas into a principled framework, reporting $17\times$

and $53\times$ speedups over cuGraph and Gunrock respectively. Our work does not modify GALA’s kernel logic; instead it addresses upstream bottlenecks that the kernel optimisations alone cannot fix.

Graph reordering. Reordering vertices to improve spatial locality is a classical technique in scientific computing and has been studied thoroughly in the context of sparse matrix operations and GPU graph traversal [10]. To the best of our knowledge, no prior work has applied it specifically to GPU Louvain implementations, making our N1 variants an original contribution to this space.

LPA-based warm starts. The Leiden algorithm [8] and parallel correlation clustering [9] both exploit movement history to warm start their iterative refinement, but neither runs LPA as a dedicated GPU preprocessing pass. Our N2 kernel is designed specifically to match GALA’s prefix-sum CSR layout and to keep the overhead well below the savings it generates.

Adaptive kernel dispatch. Choosing between different GPU kernels at runtime based on workload characteristics has proven effective for BFS [11] and triangle counting [12]. We bring the same idea to community detection by reading the degree distribution directly from the loaded graph rather than relying on compile-time heuristics.

VI. CONCLUSION

We set out to close three performance gaps that remain in GALA even after its impressive kernel-level optimisations, and we did so by building three complementary preprocessing and dispatch improvements. On the com-Amazon graph with an NVIDIA T4 GPU, degree-sorted reordering alone delivers a $1.76\times$ peak speedup in the MG pruning mode; BFS reordering reaches $1.67\times$ while keeping modularity exactly at baseline. LPA warm-start tops out at $1.38\times$ and the adaptive threshold at $1.46\times$; running all three together gives $1.35\times$ at best while tying for the highest modularity (0.9225) in the whole study.

Looking at the round-0 timing data, the picture is clear: irregular memory access is the dominant cost, and fixing it—whether through BFS or degree sorting—cuts DecideAndMove time by more than half. The adaptive threshold experiment also surfaced something instructive: com-Amazon’s degree distribution is heavily skewed, with three-quarters of vertices having degree at most 6, yet GALA’s fixed boundary of 32 was routing all of them to the costlier hash-based kernel. A simple runtime measurement corrected that immediately.

Going forward, the most pressing next step is to run these enhancements on graphs with tens or hundreds of millions of edges—LiveJournal, Orkut, and Friendster—where memory-access patterns matter even more. On the algorithm side, it would be interesting to update the adaptive threshold after each round as the active set shrinks, and to explore whether Leiden-style connectivity guarantees [8] can be woven into the LPA warm-start without adding significant overhead.

REFERENCES

- [1] V.D. Blondel, J.-L. Guillaume, R. Lambiotte, and E. Lefebvre, “Fast unfolding of communities in large networks,” *J.*

- Stat. Mech.*, vol. 2008, no. 10, p. P10008, 2008.
- [2] Z. Wang, X. Lin, X. Li, P. Wang, Z. Meng, H. Liu, C. Tian, and S. Zhong, “Swift Unfolding of Communities: GPU-Accelerated Louvain Algorithm,” in *Proc. 30th ACM SIGPLAN Symp. PPOPP*, Las Vegas, NV, USA, Mar. 2025, pp. 441–454.
 - [3] H. Lu, M. Halappanavar, and A. Kalyanaraman, “Parallel heuristics for scalable community detection,” *Parallel Comput.*, vol. 47, pp. 19–37, 2015.
 - [4] S. Ghosh, M. Halappanavar, A. Tumeo, A. Kalyanaraman, H. Lu, D. Chavarria-Miranda, A. Khan, and A. Gebremedhin, “Distributed Louvain algorithm for graph community detection,” in *Proc. IEEE IPDPS*, Vancouver, Canada, 2018, pp. 885–895.
 - [5] Md. Naim, F. Manne, M. Halappanavar, and A. Tumeo, “Community detection on the GPU,” in *Proc. IEEE IPDPS*, Orlando, FL, USA, 2017, pp. 625–634.
 - [6] A. Bhowmik and S. Vadhiyar, “HyDetect: A hybrid CPU-GPU algorithm for community detection,” in *Proc. IEEE HiPC*, Hyderabad, India, 2019, pp. 2–11.
 - [7] C. Y. Cheong, H. P. Huynh, D. Lo, and R. S. M. Goh, “Hierarchical parallel algorithm for modularity-based community detection using GPUs,” in *Proc. Euro-Par*, Aachen, Germany, 2013, pp. 775–787.
 - [8] V. A. Traag, L. Waltman, and N. J. van Eck, “From Louvain to Leiden: Guaranteeing well-connected communities,” *Sci. Rep.*, vol. 9, no. 1, p. 5233, 2019.
 - [9] J. Shi, L. Dhulipala, D. Eisenstat, J. Lacki, and V. S. Mirrokni, “Scalable community detection via parallel correlation clustering,” *Proc. VLDB Endow.*, vol. 14, no. 11, pp. 2305–2313, 2021.
 - [10] D. Merrill, M. Garland, and A. Grimshaw, “Scalable GPU graph traversal,” *ACM SIGPLAN Not.*, vol. 47, no. 8, pp. 117–128, 2012.
 - [11] H. Liu and H. H. Huang, “SIMD-X: Programming and processing of graph algorithms on GPUs,” in *Proc. USENIX ATC*, Renton, WA, USA, 2019, pp. 411–428.
 - [12] S. Pandey, X. S. Li, A. Buluc, J. Xu, and H. Liu, “H-index: Hash-indexing for parallel triangle counting on GPUs,” in *Proc. IEEE HPEC*, Waltham, MA, USA, 2019, pp. 1–7.
 - [13] J. Yang and J. Leskovec, “Defining and evaluating network communities based on ground-truth,” *arXiv:1205.6233 [cs.SI]*, 2012.
 - [14] P. Bedi and C. Sharma, “Community detection in social networks,” *Wiley Interdiscip. Rev.: Data Min. Knowl. Discov.*, vol. 6, no. 3, pp. 115–135, 2016.
 - [15] M. Cinelli, G. D. F. Morales, A. Galeazzi, W. Quattrociocchi, and M. Starnini, “The echo chamber effect on social media,” *Proc. Natl. Acad. Sci.*, vol. 118, no. 9, p. e2023301118, 2021.
 - [16] P. Expert, T. S. Evans, V. D. Blondel, and R. Lambiotte, “Uncovering space-independent communities in spatial networks,” *Proc. Natl. Acad. Sci.*, vol. 108, no. 19, pp. 7663–7668, 2011.